Chapitre 1 — Introduction à l’IA Générative Textuelle

Concepts de base • Historique • Applications

Cours — Intelligence Artificielle / NLP / LLM

Objectif : comprendre ce qu’est l’IA générative textuelle, comment elle a évolué et pourquoi les grands modèles de langage constituent une rupture importante.

Objectifs pédagogiques

Définir l'IA générative et distinguer génération de contenu et prédiction/classification.

Expliquer le fonctionnement général d'un système génératif textuel.

Situer l'IA générative textuelle dans l'histoire de l'IA et du NLP.

Comprendre l'évolution : systèmes à règles → statistiques → réseaux neuronaux → Transformers → LLM.

Identifier les principales applications de la génération de texte.

Comprendre les concepts fondamentaux : token, contexte, modèle de langage, paramètres, entraînement et inférence.

Identifier les limites : hallucinations, biais, confidentialité, coût et dépendance au contexte.

Plan du chapitre

1. Pourquoi générer du texte ?
2. Qu’est-ce que l’IA générative ?
3. IA générative textuelle : concepts fondamentaux
4. Comment un modèle génère-t-il du texte ?
5. Historique et évolution
6. Des modèles statistiques aux Transformers
7. Des Transformers aux LLM
8. Applications de la génération de texte
9. Limites et enjeux
10. Synthèse et évaluation

1. Pourquoi générer du texte ?

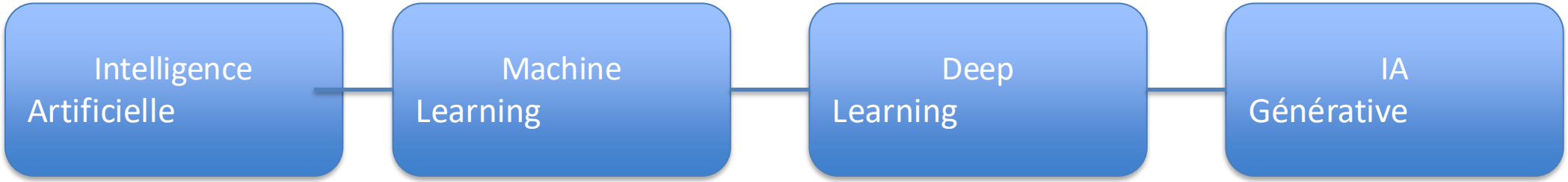
Le langage naturel est un moyen central de communication entre humains et systèmes informatiques.

Pendant longtemps, les systèmes NLP étaient principalement conçus pour analyser ou classer du texte.

La génération de texte inverse en partie la perspective : le système doit produire une séquence linguistique cohérente.

Exemples : répondre à une question, résumer un document, traduire, rédiger, compléter du code ou dialoguer.

2. IA, Machine Learning, Deep Learning et IA générative



Attention : ces catégories sont imbriquées de manière simplifiée pour l'introduction ; l'IA générative n'est pas limitée au Deep Learning, même si les modèles modernes reposent largement sur celui-ci.

3. Définition — IA générative

L’IA générative désigne les méthodes permettant à un système d’apprendre une représentation de données et de produire de nouveaux contenus à partir de cette connaissance.

Dans le cas textuel, le contenu généré est une séquence de symboles linguistiques : tokens, mots, phrases ou documents.

Le système ne se contente donc pas de sélectionner une classe existante : il construit une sortie nouvelle, token après token.

IA discriminative vs IA générative

Approche discriminative : apprendre à prédire une étiquette ou une décision à partir d'une entrée.

Exemple : texte → sentiment = positif / négatif.

Approche générative : apprendre une distribution permettant de produire de nouvelles observations.

Exemple : contexte → génération d'une suite de tokens.

Idée clé : classification = décider parmi des sorties ; génération = construire une sortie.

4. Qu’est-ce qu’un modèle de langage ?

Un modèle de langage attribue des probabilités à des séquences ou à leurs continuations.

Dans les LLM autorégressifs, le modèle estime généralement : P(token suivant | tokens précédents).

Le modèle apprend les régularités statistiques du langage à partir de données d’entraînement.

Ces régularités concernent notamment le vocabulaire, la syntaxe, les relations sémantiques et des régularités discursives.

Le principe fondamental : prédire le prochain token



Le modèle reçoit un contexte.

Il calcule une distribution de probabilités sur les tokens possibles.

Un token est sélectionné selon la stratégie de décodage.

Le nouveau token est ajouté au contexte, puis le processus recommence.

Token : l’unité manipulée par un LLM

Un token est une unité de texte utilisée par le modèle ; ce n’est pas nécessairement un mot complet.

Selon le tokenizer, un mot peut correspondre à un ou plusieurs tokens.

Les espaces, signes de ponctuation et fragments de mots peuvent également participer à la tokenisation.

La tokenisation transforme le texte en identifiants numériques exploitables par le réseau neuronal.

Conséquence : le coût, la longueur du contexte et la génération sont généralement exprimés en tokens.

Le contexte

Le contexte correspond aux tokens accessibles au modèle au moment où il produit une prédiction.

Le modèle utilise le contexte pour déterminer quelles continuations sont plausibles.

Plus le contexte pertinent est disponible, plus le modèle peut conditionner sa génération sur l'information précédente — dans la limite de sa fenêtre de contexte.

Un contexte trop long ou mal structuré peut néanmoins rendre l'information pertinente plus difficile à exploiter.

Prompt, contexte et sortie



Le prompt spécifie une tâche ou fournit un contexte.

Le modèle calcule les prochaines probabilités conditionnelles.

La sortie est produite de manière séquentielle.

Paramètres et apprentissage

Les paramètres (poids) sont les valeurs apprises par le réseau neuronal.

Pendant l'entraînement, les paramètres sont ajustés afin de réduire une fonction de perte.

Les paramètres constituent une grande partie de la capacité du modèle à représenter des régularités apprises.

Il faut distinguer les paramètres du modèle et les données fournies dans le prompt.

Entraînement vs inférence

ENTRAÎNEMENT : les paramètres sont modifiés à partir des données et d'un objectif d'apprentissage.

INFÉRENCE : les paramètres sont généralement fixés et le modèle est utilisé pour produire une sortie.

Pendant l'inférence, un prompt peut modifier fortement la sortie sans modifier les paramètres.

Le fine-tuning constitue, lui, une phase d'adaptation dans laquelle les paramètres peuvent être réentraînés ou adaptés.

Génération probabiliste

À chaque étape, le modèle produit des scores/probabilités pour les tokens candidats.

La génération peut sélectionner le token le plus probable ou utiliser une stratégie de sampling.

Des paramètres de décodage comme temperature, top-k ou top-p modifient le comportement de génération.

La génération n’est donc pas simplement une recherche d’un texte stocké dans une base de réponses.

5. Historique — Des débuts de l’IA aux systèmes symboliques

1950 — Alan Turing pose la question « Can machines think? » et propose le jeu d’imitation.

1956 — conférence de Dartmouth : naissance institutionnelle du domaine de l’Artificial Intelligence.

Premières décennies : systèmes symboliques, logique, recherche, règles et représentation explicite des connaissances.

Pour le langage, ces approches nécessitent souvent des règles nombreuses et difficiles à maintenir.

Systèmes à règles et systèmes experts

Principe : représenter explicitement des connaissances sous forme de règles.

Exemple conceptuel : SI condition ALORS conclusion.

Avantage : comportement interprétable lorsque les règles sont explicites.

Limite : faible capacité à généraliser au-delà des situations prévues et coût élevé de construction des règles.

Le passage aux approches apprenantes réduit la nécessité de programmer manuellement toutes les régularités.

Années 1990–2000 — NLP statistique

Le NLP s’oriente progressivement vers l’apprentissage à partir de corpus.

Les modèles statistiques exploitent les fréquences et cooccurrences observées dans les données.

Les modèles de langage n-gramme estiment une continuation à partir d’un contexte limité.

Exemple : $P(w_i \mid w_{i-n+1}, \ldots, w_{i-1})$.

Limite : le contexte est local et la représentation des dépendances linguistiques reste limitée.

Années 2010 — Réseaux neuronaux et représentations distribuées

Les réseaux neuronaux apprennent des représentations continues du texte.

Les embeddings représentent les mots ou tokens dans un espace vectoriel.

Les architectures RNN/LSTM introduisent une modélisation séquentielle plus flexible.

Les mécanismes d’attention permettent de mieux exploiter des éléments pertinents du contexte.

2017 — Le Transformer

Le Transformer, introduit par Vaswani et al. dans “Attention Is All You Need”, repose principalement sur des mécanismes d’attention.

Le self-attention permet de calculer des relations entre les différents tokens d’une séquence.

L’architecture facilite la parallélisation de l’entraînement par rapport aux modèles récurrents.

Le Transformer devient ensuite l’architecture dominante des grands modèles de langage.

Intuition du Self-Attention

Pour représenter un token, le modèle peut pondérer l'information provenant d'autres tokens du contexte.

L'attention permet donc de construire une représentation dépendant du contexte.

Exemple : dans une phrase, le mot "il" peut nécessiter de considérer des mots précédents pour interpréter son référent.

Cette capacité à établir des dépendances contextuelles est centrale dans les architectures Transformer.

2018 — GPT et le Generative Pre-Training

GPT-1 popularise une approche fondée sur le generative pre-training.

Un modèle est d’abord pré-entraîné sur de grandes quantités de texte non annoté.

La connaissance linguistique acquise peut ensuite être réutilisée pour différentes tâches.

Cette idée contribue au passage des modèles spécialisés vers des modèles plus généraux.

2019–2020 — GPT-2, GPT-3 et changement d’échelle

GPT-2 augmente fortement l’échelle des modèles et des données.

GPT-3 pousse davantage cette logique d’échelle et met en avant le few-shot learning.

L’utilisation d’exemples dans le contexte permet d’orienter le modèle vers une tâche sans nécessairement modifier ses poids.

La notion de Large Language Model (LLM) devient centrale dans l’évolution récente du NLP.

De GPT aux LLM modernes

Les LLM modernes reposent généralement sur des architectures Transformer et un pré-entraînement à grande échelle.

Des étapes supplémentaires peuvent être utilisées : instruction tuning, préférence humaine, RLHF/DPO, adaptation spécialisée, etc.

Les modèles modernes peuvent intégrer texte, code, images ou audio selon leur architecture.

Dans ce chapitre, nous nous concentrons sur la génération textuelle.

6. Applications de la génération de texte

Dialogue et assistants conversationnels

Génération et reformulation de contenu

Résumé automatique

Traduction automatique

Question answering

Extraction et transformation de texte

Génération de code

Éducation et tutorat

Aide à la recherche et à l’analyse documentaire

Création de données synthétiques

Application 1 — Génération et rédaction

Rédaction de brouillons : rapports, courriers, descriptions, comptes rendus.

Reformulation selon un niveau de langue ou un style demandé.

Expansion : transformer une idée courte en texte structuré.

Compression : produire une version courte d’un contenu.

Point de vigilance : une génération fluide n’implique pas automatiquement une information exacte.

Application 2 — Résumé automatique

Entrée : document ou ensemble de documents.

Sortie : représentation textuelle plus courte conservant les informations importantes.

Deux grandes familles : résumé extractif et résumé abstractive.

Les LLM sont particulièrement adaptés au résumé abstractive, mais peuvent produire des informations absentes de la source.

Pour un usage académique, la vérification par rapport au document source reste essentielle.

Application 3 — Traduction

Le modèle génère une séquence dans une langue cible conditionnée par le texte source.

Les LLM peuvent exploiter des représentations multilingues apprises durant le pré-entraînement.

La qualité dépend de la langue, du domaine, du contexte et des données d’apprentissage.

Une traduction générée doit être vérifiée dans les contextes professionnels, juridiques ou scientifiques sensibles.

Application 4 — Question Answering

L’utilisateur fournit une question et un contexte éventuel.

Le modèle génère une réponse en langage naturel.

Dans un système RAG, des documents externes peuvent être récupérés avant la génération.

Distinction importante : un LLM seul génère à partir de ses paramètres + contexte ; un système RAG ajoute une étape de récupération documentaire.

Application 5 — Génération de code

Génération de fonctions et de fragments de programme.

Explication de code existant.

Transformation d’un langage de programmation vers un autre.

Détection et correction de problèmes dans du code.

La validation par exécution, tests et revue humaine reste nécessaire.

Application 6 — Éducation

Génération d’exercices et de questions.

Explication d’un concept à plusieurs niveaux de difficulté.

Tutorat conversationnel.

Création de résumés et fiches de révision.

Adaptation linguistique et pédagogique du contenu.

Limite : le modèle peut produire une explication convaincante mais incorrecte.

Application 7 — Entreprise et documentation

Rédaction de comptes rendus.

Synthèse de réunions.

Recherche conversationnelle dans la documentation.

Génération de réponses pour le support client.

Classification + génération pour automatiser des workflows textuels.

RAG et outils externes permettent de connecter le LLM à des sources internes.

Application 8 — Recherche scientifique

Assistance à la recherche bibliographique et à la synthèse.

Reformulation et traduction scientifique.

Génération de pistes expérimentales ou de code.

Extraction structurée d’informations depuis des articles.

Important : distinguer assistance à la recherche et validation scientifique.

7. Limites de l’IA générative textuelle

Hallucinations : génération d’informations fausses ou non étayées.

Biais : reproduction ou amplification de biais présents dans les données.

Contexte limité : difficulté à exploiter toute l’information pertinente.

Sensibilité au prompt et au contexte.

Confidentialité : risque lors de l’envoi de données sensibles à un service.

Coût computationnel et énergétique.

Difficulté d’expliquer précisément pourquoi une réponse particulière a été produite.

Hallucination : un concept fondamental

Une hallucination est une sortie qui peut être linguistiquement plausible tout en étant factuellement incorrecte ou non supportée.

La fluidité linguistique n’est pas une preuve de vérité.

Les causes peuvent être liées aux données, au contexte, à l’objectif de génération et au mécanisme probabiliste.

Stratégies de réduction : RAG, citations vérifiables, outils, contraintes, tests et validation humaine.

Enjeux éthiques et de sécurité

Protection des données personnelles et confidentielles.

Biais et discrimination.

Désinformation et génération de contenu trompeur.

Droits d’auteur et provenance des données.

Sécurité des systèmes connectés à des outils externes.

Responsabilité humaine lors de l’utilisation dans des domaines à enjeux.

Distinctions à maîtriser

IA générative ≠ uniquement ChatGPT.

LLM ≠ chatbot : un chatbot est une application ; un LLM est un modèle pouvant être utilisé par différentes applications.

Prompt ≠ entraînement : un prompt modifie le contexte fourni au modèle.

Pré-entraînement ≠ inférence.

Génération ≠ vérité.

Token ≠ mot.

Paramètres du modèle ≠ contexte de la conversation.

Synthèse du chapitre

L’IA générative produit de nouveaux contenus à partir de modèles appris sur des données.

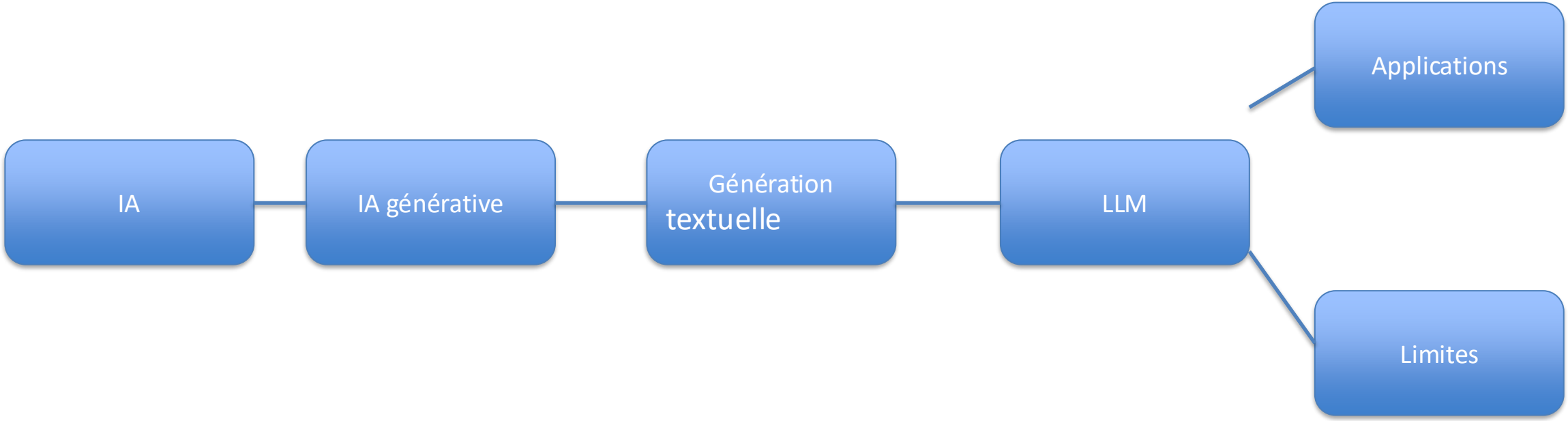
En génération textuelle, les LLM modélisent des séquences de tokens et génèrent généralement de manière autorégressive.

L’histoire du domaine va des systèmes symboliques et statistiques vers les réseaux neuronaux puis les Transformers et LLM.

Le Transformer constitue une étape majeure grâce au self-attention.

Les LLM modernes ont de nombreuses applications mais restent sujets aux hallucinations, biais et limites contextuelles.

Carte conceptuelle — Chapitre 1



Progression historique : symbolique → statistique → neural → Transformer → LLM

Questions de compréhension

1. Quelle différence fondamentale existe entre une approche discriminative et une approche générative ?
2. Pourquoi un LLM manipule-t-il des tokens plutôt que directement des mots ?
3. Qu’est-ce qu’un modèle de langage ?
4. Expliquez la génération autorégressive.
5. Quelle est la différence entre entraînement et inférence ?
6. Pourquoi le Transformer a-t-il constitué une rupture importante pour le NLP ?
7. Quelle différence entre un LLM et un chatbot ?
8. Donnez quatre applications de la génération textuelle.
9. Qu’est-ce qu’une hallucination ?
10. Pourquoi une réponse fluide ne garantit-elle pas sa véracité ?

À retenir

L’IA générative textuelle est l’aboutissement d’une longue évolution de l’IA et du NLP.

Les LLM modernes reposent sur des modèles capables de modéliser et générer des séquences de tokens.

Le Transformer et le self-attention ont profondément transformé la modélisation du langage.

La génération textuelle est extrêmement polyvalente, mais elle ne garantit ni exactitude ni neutralité.

Comprendre les concepts fondamentaux est indispensable avant d’aborder le prompt engineering, le fine-tuning, le RAG et les agents.